FPT Education
FPT UNIVERSITY

Chapter 8: Device Driver Basics

Drivers • Modules • Kernel Interaction

FPT Education
FPT UNIVERSITY

Learning Objectives

- Understand device driver concept
- Learn kernel module mechanism
- Understand driver operations
- Interact with hardware via driver

FPT Education
FPT UNIVERSITY

Role of Device Drivers

Device Driver là lớp phần mềm trong kernel giúp Linux điều khiển phần cứng mà ứng dụng không cần biết chi tiết phần cứng hoạt động như thế nào.

application thông thường không điều khiển trực tiếp hardware.

- Abstract hardware complexity from applications

với Linux driver, application có thể chỉ thấy: /dev/ttyS0 và sử dụng: open(),read(),write(), close()

- Provide unified interface via /dev

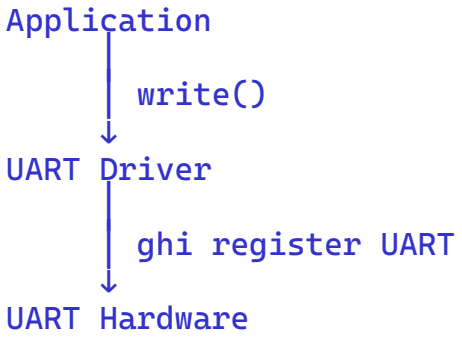
Nhiều thiết bị được Linux biểu diễn dưới dạng device file trong: /dev VD: /dev/ttyS0, /dev/i2c-1 ...

- Ensure safe kernel-hardware interaction

User application thông thường không được phép tự ý truy cập: CPU registers, MMIO registers, interrupt controller,DMA controller,physical memory

 Drivers act as translators between user space requests and hardware operations

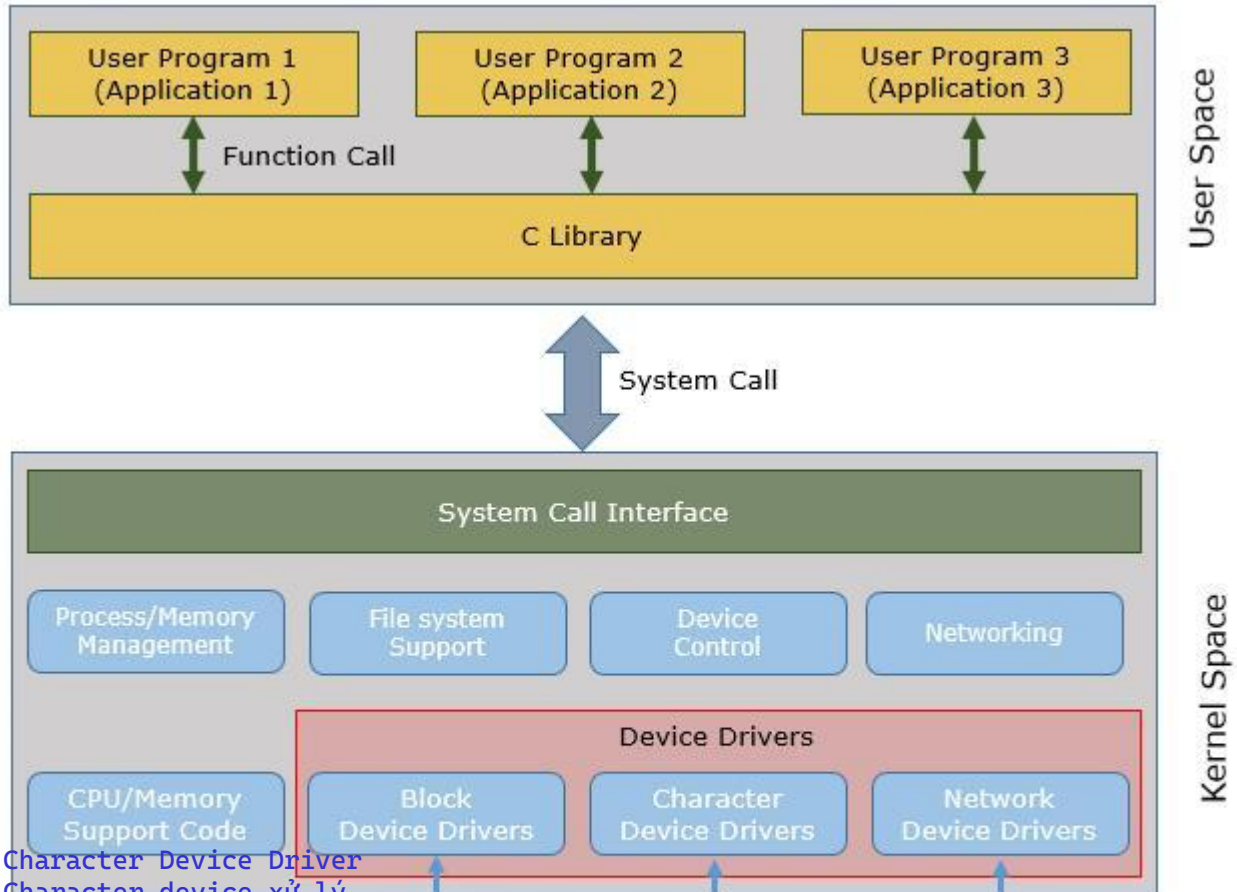
Ví dụ một chương trình muốn gửi dữ liệu qua UART:
write(fd, "Hello", 5);
Ứng dụng chỉ gọi write(). Nó không cần biết:
thanh ghi UART nằm ở địa chỉ nào,
bit nào kích hoạt transmitter,
FIFO của UART hoạt động thế nào,
interrupt nào được sử dụng.
Các chi tiết đó được UART driver xử lý.



VD:
write(fd, buffer, 5);
Driver dịch (translator) yêu cầu đó thành:
1. Kiểm tra UART có sẵn không
2. Copy dữ liệu vào buffer
3. Cấu hình UART nếu cần
4. Ghi dữ liệu vào TX FIFO
5. Enable TX interrupt
6. Đợi hardware truyền
7. Xử lý interrupt
8. Báo kết quả cho application

FPT Education
FPT UNIVERSITY

Device Drivers



Block Device Driver
Hard disk,SSD,SD card
SD,USB, ...
Device file:
/dev/sda
/dev/sdb
/dev/mmcblk0

Character Device Driver
Character device xử lý dữ liệu dưới dạng dòng byte.
UART,keyboard,mouse
GPIO,I2C,SPI,sensor
device file:
/dev/ttyS0
/dev/i2c-1
/dev/input/event0

Network driver điều khiển:
Ethernet, Wi-Fi,CAN, ...
eth0, wlan0

bare-metal MCU

Bare-metal:

Application
    ↓
HAL / Register
    ↓
Hardware

Linux

Application
    ↓
System Call
    ↓
Kernel
    ↓
Device Driver
    ↓
Hardware

Application Linux thông thường không trực tiếp chạm vào register hardware.

FPT Education
FPT UNIVERSITY

Loadable Kernel Modules

cơ chế cho phép Linux nạp thêm hoặc gỡ bớt một phần mã kernel khi hệ thống đang chạy, thay vì phải biên dịch lại toàn bộ kernel. Thường được dùng để triển khai device driver, filesystem hoặc một số thành phần mạng.

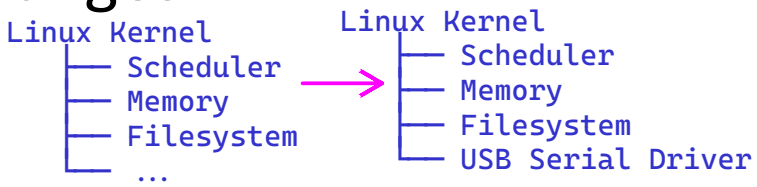
- LKMs allow runtime extension of kernel functionality
- Avoid full kernel rebuild and reboot
- Common for drivers, filesystems, network protocols
  - LKM enabled:
    + Dynamic loading/unloading
    + Easier debugging and updates
  - LKM disabled:
    + More secure and lightweight
    + Requires kernel rebuild for changes

build dưới dạng module

Kernel Linux phải được cấu hình để hỗ trợ module.

CONFIG_MODULES=y
⇒ kernel hỗ trợ: load module unload module
VD: insmod my_driver.ko
hoặc modprobe my_driver
gỡ bằng: rmmod my_driver

VD: ban đầu kernel chưa có driver cho một thiết bị USB
Sau đó ta nạp:
modprobe usbserial



Linux kernel có thể chứa mã theo hai cách:
Cách 1: Built-in
Code được biên dịch trực tiếp vào kernel
↓
nằm trong vmlinux / Image / zImage

Cách 2: Module
Code được biên dịch thành file riêng
↓
*.ko (kernel object)

Khi cần, kernel có thể nạp file .ko vào bộ nhớ
mà không cần reboot
Kernel đang chạy
+
driver.ko
↓
Kernel có thêm chức năng mới

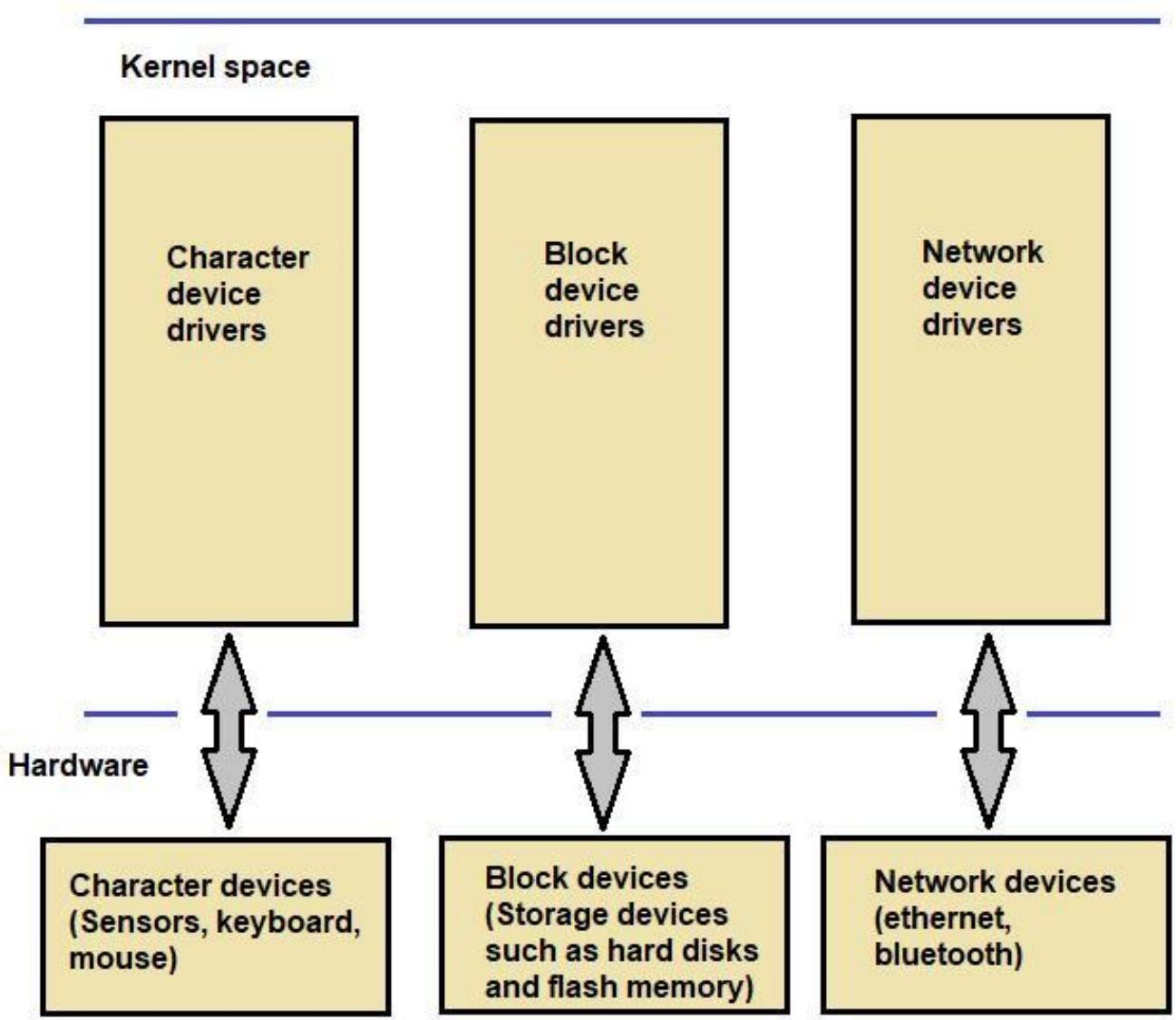
khi phát triển driver.
viết một driver: my_driver.c
driver được build trực tiếp vào kernel:
Sửa driver.c
↓
Build kernel
↓
Tạo Image/zImage
↓
Copy kernel sang board
↓
Reboot
↓
Test

*insmod
VD: sudo insmod my_driver.ko
Nạp trực tiếp một file .ko: Đơn giản nhưng không tự
xử lý dependency.
driver_A
↓ depends on
driver_B
dùng insmod driver_A.ko mà driver_B
chưa được load thì có thể lỗi.

*modprobe:
VD: sudo modprobe driver_A
modprobe sẽ kiểm tra dependency và tự load:
driver_B
↓
driver_A
⇒ ưu tiên dùng modprobe

dùng LKM: Không phải reboot hệ thống.
Sửa driver.c
↓
make
↓
my_driver.ko
↓
rmmod my_driver
↓
insmod my_driver.ko
↓
Test

Device Types



FPT Education
FPT UNIVERSITY

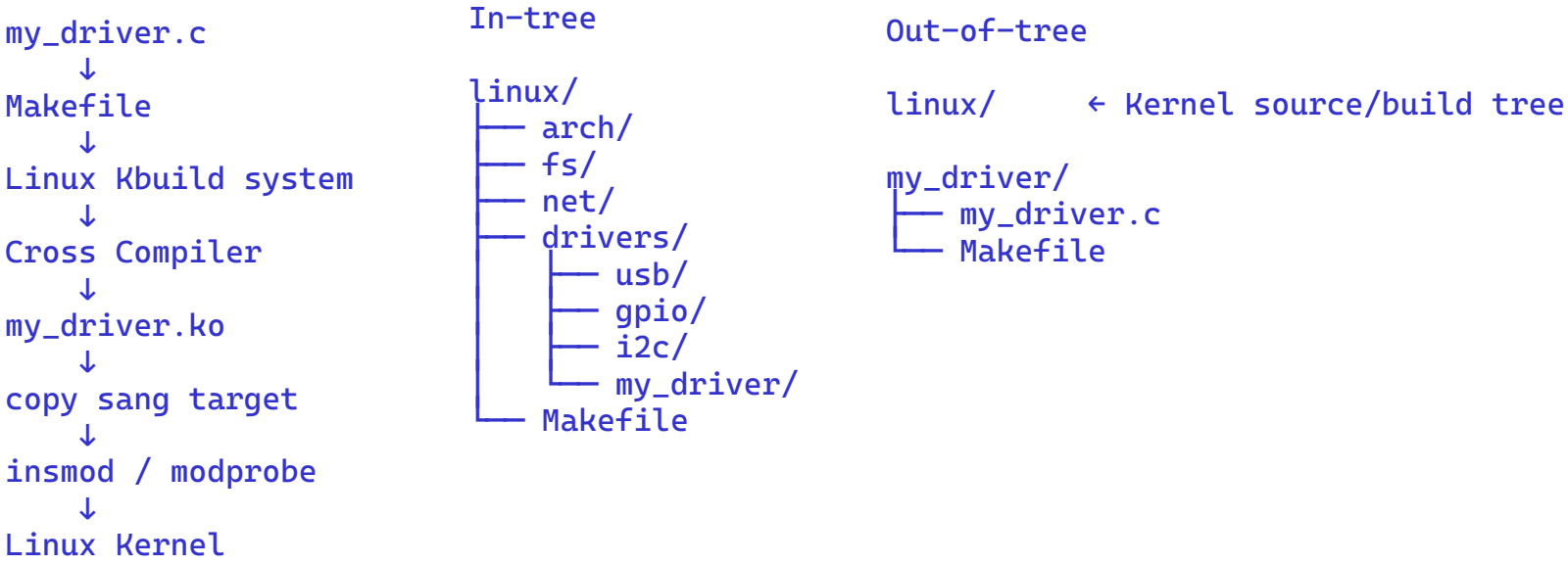
Build Infrastructure

Có hai cách đặt source code driver.

- In-tree or out-of-tree build
- Use kernel headers and Makefile
- Standard module build command

```
make -C $KDIR \
    ARCH=arm \
    CROSS_COMPILE=arm-linux-gnueabihf- \
    M=$(PWD) \
    modules
```

In-tree: driver nằm bên trong Linux kernel source tree
Out-of-tree: source driver nằm bên ngoài kernel source



Kernel module không phải một chương trình user-space bình thường.
Không thể đơn giản: gcc my_driver.c -o my_driver
vì module cần:

kernel headers,
kernel configuration,
compiler flags của kernel,
architecture-specific definitions,
symbol information,
Kbuild rules,
module metadata.

Do đó ta phải build thông qua Linux Kernel Build System – Kbuild.

FPT Education
FPT UNIVERSITY

Module Utilities

- insmod: Loads a specific kernel module using its complete file path.
- lsmod: Displays a formatted list of currently loaded modules and their dependency chains.
- rmmod: Removes a kernel module from the running kernel
  - The module cleanup function is executed on removal
  - Registered using the module_exit() macro
  - Frees resources (memory, devices, interrupts)

module_init() dùng để khởi tạo/cấp phát tài nguyên,
còn module_exit() dùng để dọn dẹp và trả lại tài nguyên.

Module Utilities (2)

- modprobe: Intelligently loads modules while automatically resolving and loading required dependencies.
- depmod: Generates the modules.dep database file that modprobe uses to determine dependencies.
- modinfo: Displays useful module metadata and accepted command-line parameters.

FPT Education
FPT UNIVERSITY

Driver Methods

cách Linux device driver nối các thao tác file từ user space với các hàm bên trong driver.

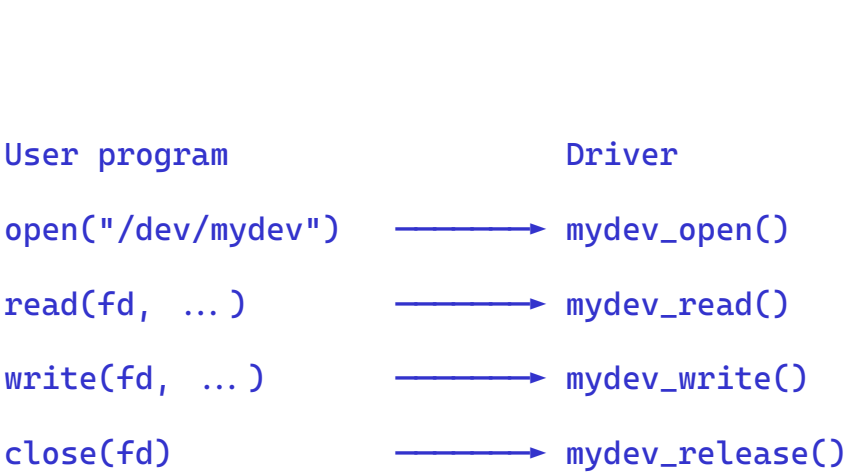
- Device drivers use the file_operations structure to link file system requests directly to custom driver functions

file_operations = cầu nối giữa system call của user và hàm của driver.

```
static struct file_operations fops = {
    .open = mydev_open,
    .read = mydev_read,
    .write = mydev_write,
    .release = mydev_release,
};
```

- Use unlocked_ioctl instead of ioctl → WHY?

ioctl() dùng cho các lệnh điều khiển thiết bị mà read/write không diễn tả thuận tiện.
Ví dụ: Turn LED ON, Set baud rate, Reset device, Set operating mode
.unlocked_ioctl = mydev_ioctl,
API .ioctl cũ từng liên quan đến cơ chế Big Kernel Lock (BKL), tức kernel có thể khóa rất rộng khi xử lý ioctl. Điều này làm giảm khả năng chạy song song. unlocked_ioctl bỏ cơ chế khóa toàn cục đó.




```
unlocked_ioctl
      ↓
Driver function
      ↓
Driver tự quản lý locking nếu cần
```

Ví dụ driver có dữ liệu dùng chung thì có thể tự bảo vệ bằng:

```
mutex_lock(&my_lock);

/* access shared hardware/data */

mutex_unlock(&my_lock);
```

Ví dụ nếu chương trình user chạy:

```
fd = open("/dev/mydev", O_RDWR);
write(fd, "Hello", 5);
close(fd);
```

thì driver tương ứng sẽ thực hiện gần như:

```
open()
   ↓
mydev_open()

write()
   ↓
mydev_write()

close()
   ↓
mydev_release()
```

release trong driver tương ứng với việc đóng file descriptor, thường xảy ra khi user gọi: close(fd);

FPT Education
FPT UNIVERSITY

Driver Method (2)

- Device Numbers: Drivers are identified by the kernel using allocated major and minor device numbers. Device node là file đặc biệt trong /dev, ví dụ: /dev/mydev0
- Device Nodes: Special files usually located in /dev that act as pointers to the device driver.
  - alloc_chrdev_region(): dynamic major allocation (preferred) Dùng để yêu cầu kernel tự động cấp major number.
  - register_chrdev_region(): static major (fixed, manual)
- Dynamic avoids conflicts across multiple drivers
- Static suitable only for controlled or legacy systems

Mỗi thiết bị kiểu character device được kernel nhận diện bằng một cặp số:
Major number : Minor number
Major: thường xác định driver nào quản lý thiết bị.
Minor: phân biệt các thiết bị/instance khác nhau dùng cùng driver.
VD: Major = 240
/dev/mydev0 → 240:0
/dev/mydev1 → 240:1
/dev/mydev2 → 240:2
Cả 3 thiết bị dùng cùng driver 240, nhưng có minor khác nhau.
Major → Driver nào?
Minor → Thiết bị nào trong driver đó?

Ứng dụng user-space thao tác với thiết bị thông qua file này:
open("/dev/mydev0", ...);
read(...);
write(...);

VD: cấp major number tự động
dev_t dev;
alloc_chrdev_region(&dev, 0, 1, "mydev");
Sau đó lấy:
MAJOR(dev);
MINOR(dev);

chỉ định major number cố định:
dev_t dev = MKDEV(240, 0);
register_chrdev_region(dev, 1, "mydev");

Dynamic	Static
`alloc_chrdev_region()`	`register_chrdev_region()`
Kernel tự cấp major	Người lập trình tự chọn major
Ít nguy cơ xung đột	Có thể bị trùng
Được khuyến nghị	Chủ yếu dùng cho hệ thống cố định/legacy

Application
↓
/dev/mydev0
↓
Major : Minor
↓
Kernel
↓
Device Driver
↓
file_operations
↓
mydev_open()
mydev_read()
mydev_write()

FPT Education
FPT UNIVERSITY

Driver Method (3)

cách Linux tạo các device node trong /dev một cách tự động sau khi driver/thiết bị xuất hiện.

- Device nodes created dynamically by udev
- Kernel exports device info via /sys (sysfs) or /proc/devices
- Desktop Linux: uses udev (dynamic device management, requires systemd)
- Embedded Linux: udev often too heavy for resource-constrained systems
- Common alternatives:
  - mdev (BusyBox): lightweight, simple rules
  - eudev: standalone udev (no systemd dependency)
- Static /dev nodes may still be used in minimal systems

Kernel cung cấp thông tin thiết bị qua /sys

```
ls /sys/class/tty/
```

udev đọc thông tin trong /sys để biết:
thiết bị nào vừa xuất hiện
major/minor number
loại thiết bị
driver tương ứng

Một hệ thống embedded cực kỳ đơn giản thậm chí có thể tạo sẵn device node: mknod /dev/mydev c 240 0 lưu trực tiếp trong root filesystem:

```
/dev/
├── console
├── null
├── ttyS0
└── mydev
```

Device node được tạo động bởi udev
Ví dụ khi cắm USB hoặc khi driver đăng ký một character device,
Linux có thể tự tạo:

/dev/ttyUSB0
/dev/sda
/dev/input/event0

Thay vì người dùng phải tự chạy:
mknod /dev/mydev c 240 0
thì udev sẽ tạo device node tự động.

cat /proc/devices
hiển thị các major number của character/block device đã
đăng ký, ví dụ:
Character devices:
  1 mem
  4 tty
  5 /dev/tty
189 usb

/sys   → thông tin chi tiết về device
/proc/devices → danh sách major number đã đăng ký

Driver / Device xuất hiện
        ↓
Kernel phát hiện thiết bị
        ↓
Kernel gửi uevent
        ↓
udev nhận sự kiện
        ↓
Tạo /dev/xxx

Finding Out About Drivers at Runtime

cách kiểm tra các driver và thiết bị đang tồn tại khi Linux đang chạy.

- Read /proc/devices to list currently loaded character and block device drivers.

```
cat /proc/devices
```

- Use ifconfig or ip commands to list network devices.

```
ip link
ifconfig -a
```

- Explore sysfs (/sys/devices, /sys/class, /sys/block) to see the kernel's view of devices, driver classes, and their relationships.

```
ls /sys/class
ls /sys/class/net
ls /sys/block
```

FPT Education
FPT UNIVERSITY

cách Linux tìm và tự động nạp đúng driver cho một phần cứng.

Finding the Right Device Driver

Kiểm tra kernel đã có driver chưa. Với driver có sẵn trong Linux kernel, có thể tìm qua: make menuconfig

- Start by searching the mainline Linux kernel (make menuconfig) or vendor-specific kernels
- Check the hardware manufacturer's website or open-source community forums for support
  - `modalias` describes a device identity for automatic driver matching.
  - `MODULE_DEVICE_TABLE()` exports driver match tables into module metadata.
  - `udev` reads `modalias` and triggers `modprobe` to load the right driver. tự động nạp driver. Khi kernel phát hiện thiết bị:
  - `udevadm info` helps inspect which driver is bound to a device. giúp xem các thuộc tính mà udev biết về thiết bị.

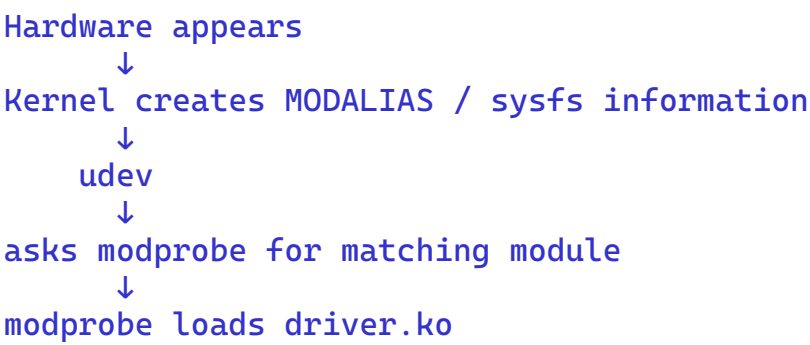
```
udevadm info /dev/sda
udevadm info --query=all --path=/sys/class/net/eth0
```

MODULE_DEVICE_TABLE() – khai báo thiết bị driver hỗ trợ.
Trong source driver thường có một bảng ID:

```
static const struct pci_device_id my_ids[] = {
    { PCI_DEVICE(0x1234, 0x5678) },
    { }
};

MODULE_DEVICE_TABLE(pci, my_ids);
```

Macro này đưa thông tin matching vào metadata của module .ko.
Sau đó depmod dùng metadata đó để tạo các alias mà modprobe có thể tra cứu.



FPT Education
FPT UNIVERSITY

điều khiển phần cứng trực tiếp từ User Space trong Linux mà không cần tự viết Kernel Driver.

Device Drivers in User Space

- Control hardware directly from user space using generic kernel drivers without writing custom kernel code.
- GPIO & LEDs: Accessed and controlled via the sysfs interface (/sys/class/gpio and /sys/class/leds).
- I2C & SPI: Controlled via generic device nodes (e.g., /dev/i2c-0, /dev/spidev1.0) and standard user space APIs.

VD: với GPIO kiểu sysfs truyền thống:

```
echo 17 > /sys/class/gpio/export
echo out > /sys/class/gpio/gpio17/direction
echo 1 > /sys/class/gpio/gpio17/value
```

export → yêu cầu kernel cho phép sử dụng GPIO17
direction → cấu hình GPIO17 là output
value = 1 → đưa chân GPIO lên mức HIGH

Với LED đã được kernel đăng ký, có thể gặp:

```
/sys/class/leds/
    led0/
        brightness
        trigger
```

bật LED: echo 1 > /sys/class/leds/led0/brightness

*/sys/class/gpio là interface cũ và đã bị deprecated
trên các Linux kernel mới. Hiện nay thường dùng GPIO c
haracter device + libgpiod, ví dụ gpiodetect, gpioinfo, gpioset, gpioget.

Device Drivers in User Space (2)

- GPIO: Control pins directly via the sysfs interface
  - Example (Exporting a pin): echo 53 > /sys/class/gpio/export
- LEDs: Manage brightness and trigger events via sysfs
  - Example (Setting a timer trigger): echo timer > trigger
- I2C: Communicate using standard user space APIs and ioctl
  - Example (C code to set slave address): ioctl(f, I2C_SLAVE, I2C_ADDRESS);
- SPI: Access full-duplex communication via generic device nodes like /dev/spidev1.0

FPT Education
FPT UNIVERSITY

Writing a Kernel Device Driver

- Necessary when user space drivers are insufficient for complex hardware
- Interfaces can be designed using sysfs (preferred), ioctl, mmap, debugfs, or netlink
- Requires defining file operations (open, read, write, release) and managing module initialization and exit functions

```
Các cách để user space giao tiếp với driver
sysfs: phù hợp để đọc/ghi các thuộc tính đơn giản của thiết bị,
ví dụ trạng thái LED, tốc độ, mode.
ioctl(): gửi các lệnh điều khiển phức tạp cho driver.
mmap(): ánh xạ vùng nhớ của thiết bị/kernel vào user space để truy cập hiệu quả hơn.
debugfs: chủ yếu phục vụ debug, không nên xem là API ổn định cho ứng dụng.
netlink: giao tiếp hai chiều giữa kernel và user space,
thường dùng cho networking hoặc event/message.
```

FPT Education
FPT UNIVERSITY

Writing a Kernel Device Driver

```
#include <linux/module.h>
#include <linux/init.h>
static int __init mydrv_init(void)
{
    pr_info("mydrv: init\n");
    return 0;
}
static void __exit mydrv_exit(void)
{
    pr_info("mydrv: exit\n");
}
module_init(mydrv_init);
module_exit(mydrv_exit);
MODULE_LICENSE("GPL");
MODULE_AUTHOR("Your Name");
MODULE_DESCRIPTION("Minimal driver skeleton");
```

hàm được gọi khi module được nạp vào kernel.

return 0 nghĩa là initialization thành công. Nếu trả về giá trị âm như -ENOMEM, -EBUSY thì việc load module thất bại.

chạy khi module bị gỡ khỏi kernel

FPT Education
FPT UNIVERSITY

Discovering the Hardware Configuration

- The kernel matches drivers to hardware components using identifiers
- Device Trees: The modern standard for describing hardware components dynamically. Linkage is done through the compatible property
- Platform Data: A fallback method using C structures to describe hardware in older SoCs without Device Tree support

```
dts:
led_controller {
    compatible = "vinh,my-led";
};

trong driver:
static const struct of_device_id my_of_match[] = {
    { .compatible = "vinh,my-led" },
    { }
};

MODULE_DEVICE_TABLE(of, my_of_match);

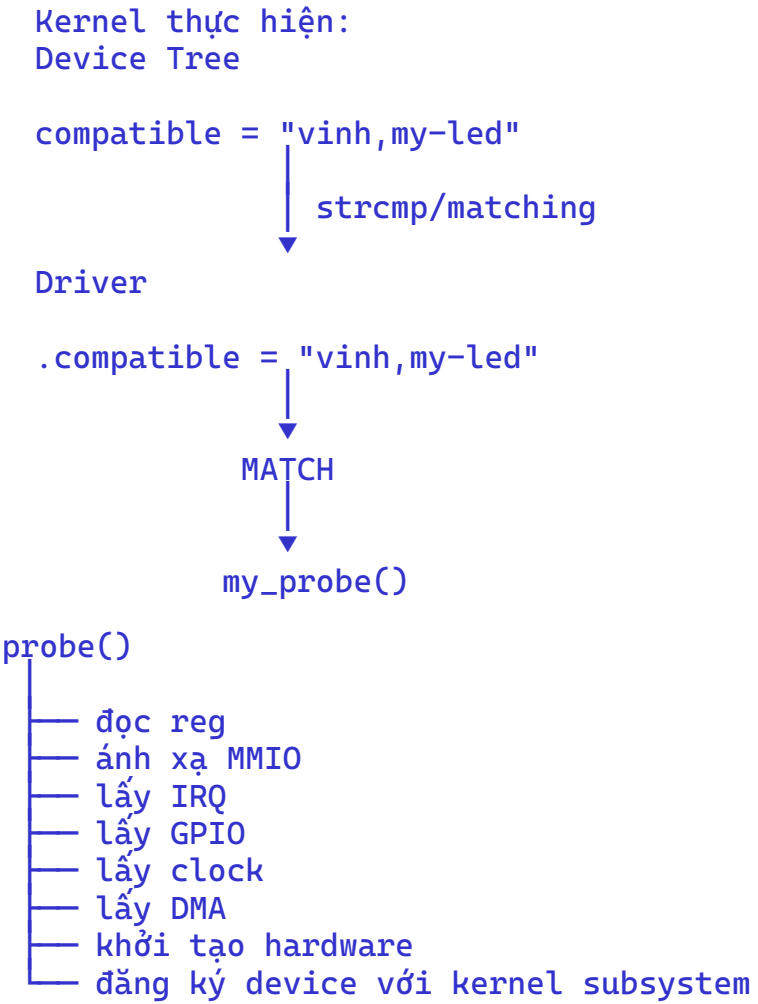
static struct platform_driver my_driver = {
    .probe = my_probe,

    .driver = {
        .name = "my_led_driver",
        .of_match_table = my_of_match,
    },
};


xác định Device Tree và driver đã match:
static int my_probe(struct platform_device *pdev)
{
    printk(KERN_INFO "My device detected!\n");

    return 0;
}


kiểm tra Device Tree đang chạy: ls /sys/firmware/devicetree/base
Xem model board: cat /sys/firmware/devicetree/base/model
Tìm tất cả compatible trong hệ thống:
find /sys/firmware/devicetree/base \
     -name compatible \
     -exec sh -c 'echo "——— $1"; tr "\0" "\n" < "$1"' _ {} \;
```



```c
#include <linux/module.h>
#include <linux/platform_device.h>
#include <linux/of.h>

static int demo_probe(struct platform_device *pdev)
{
    pr_info("DEMO: probe() called!\n");
    pr_info("DEMO: Device Tree matched successfully!\n");

    return 0;
}

static int demo_remove(struct platform_device *pdev)
{
    pr_info("DEMO: remove() called!\n");
    return 0;
}

static const struct of_device_id demo_of_match[] = {
    { .compatible = "vinh,demo-device" },
    { }
};

MODULE_DEVICE_TABLE(of, demo_of_match);

static struct platform_driver demo_driver = {
    .probe  = demo_probe,
    .remove = demo_remove,

    .driver = {
        .name = "demo_driver",
        .of_match_table = demo_of_match,
    },
};

module_platform_driver(demo_driver);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Demo");
MODULE_DESCRIPTION("Device Tree matching demo");
```

Device Tree Matching (Driver Side)

```
#include <linux/module.h>
#include <linux/of.h>
#include <linux/platform_device.h>

static const struct of_device_id mydrv_of_match[] = {
    { .compatible = "vendor,mydevice" },
    { /* sentinel */ }
};
MODULE_DEVICE_TABLE(of, mydrv_of_match);
```

- Driver declares supported devices via compatible strings
- Kernel matches DT node ↔ driver using of_match_table

FPT Education
FPT UNIVERSITY

Using of_match_table in Driver

```
static struct platform_driver mydrv = {
    .driver = {
        .name = "mydrv",
        .of_match_table =
OF_MATCH_PTR(mydrv_of_match),
    },
    .probe  = mydrv_probe,
    .remove = mydrv_remove,
};

module_platform_driver(mydrv);
```

Summary

- Device Categories: Linux groups hardware into Character devices (unbuffered I/O like serial ports), Block devices (buffered mass storage), and Network devices (accessed via sockets)
- User Space Control: You can often manage peripherals like GPIO, LEDs, I2C, and SPI directly from user space programs without needing to write any custom kernel code
- Kernel Drivers: If user space control isn't enough, you can write custom kernel modules that register devices and define file operations (like open, read, and write)
- Hardware Binding: The kernel uses structures like Device Trees or platform data to map physical hardware components to the correct drivers during boot